

Practical Docker & Docker Compose Cheat Sheet for Developers

Only the high-frequency commands and workflows you'll actually use day to day. Every command sits in its own code block so you can copy it with one click.

1. Daily Developer Routine

Morning: start work

Launch the whole stack (app + database) in the background:

```
docker compose up -d
```

Morning: code or dependencies changed

Pulled new code, edited the Dockerfile, or changed `requirements.txt` ? Rebuild the image and relaunch:

```
docker compose up -d --build
```

Evening: finish work

Stop and remove the containers and network (named volumes, such as your database data, are kept):

```
docker compose down
```

When	Command	Why
Start of day	<code>docker compose up -d</code>	Fast start using existing images
After code/dependency changes	<code>docker compose up -d --build</code>	Forces a rebuild so changes are included
End of day	<code>docker compose down</code>	Frees ports and memory

2. Most Used Docker & Docker Compose Commands

Docker Compose commands

Launch all services in the background

```
docker compose up -d
```

Reads `docker-compose.yml`, creates the network, and starts every service detached (`-d`). *Use it:* at the start of every work session.

Rebuild the image with updated code and launch

```
docker compose up -d --build
```

Rebuilds images from the Dockerfile before starting. *Use it:* after changing source code baked into the image, the Dockerfile, or dependency files.

Stop and tear down all services and the network

```
docker compose down
```

Removes the containers and network created by `up`. Your named volumes (DB data) survive. *Use it:* at the end of the day or before a clean restart.

Check the status of the Compose stack

```
docker compose ps
```

Shows each service, whether it's running or exited, and its port mappings. *Use it:* to confirm everything started correctly.

View combined logs of web and DB

```
docker compose logs
```

Prints logs from all services together. Add `-f` to follow live output. *Use it:* to debug startup errors across the whole stack.

Plain Docker commands

List all active containers

```
docker ps
```

Shows running containers with their names, IDs, images, and ports. Add `-a` to include stopped ones. *Use it:* to find a container's name for the commands below.

Inspect logs of a specific container

```
docker logs <container_name>
```

Prints that container's output. Add `-f` to follow it live. *Use it:* when one container is crashing or misbehaving.

Open an interactive shell inside a container

```
docker exec -it <container_name> sh
```

Gives you a terminal inside the running container (use `bash` instead of `sh` if the image includes it). Type `exit` to leave. *Use it:* to inspect files, run one-off scripts, or test the DB connection from the inside.

Build a standalone Docker image

```
docker build -t <name>:<tag> .
```

Builds an image from the Dockerfile in the current folder and names it (for example `my-app:1.0`). *Use it:* when you need a single image without Compose, such as in CI or before pushing to a registry.

Run a standalone container

```
docker run -d -p 8000:8000 -e APP_ENV=production <image_name>
```

Flag	Meaning
<code>-d</code>	Run in the background
<code>-p 8000:8000</code>	Map <code>HOST_PORT:CONTAINER_PORT</code> so the app is reachable at <code>localhost:8000</code>
<code>-e APP_ENV=production</code>	Set an environment variable inside the container

Use it: to run a single container without a Compose file.



3. Workflow 1: Dockerizing a New Project From Scratch

Example: a Python web app with a PostgreSQL database.

```
my-project/
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
└── .env
```

Step 1: Write the Dockerfile

Create a file named exactly `Dockerfile` in the project root:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

Instruction	What it does
<code>FROM</code>	Starting base image. <code>python:3.12-slim</code> is a small Linux with Python preinstalled.
<code>WORKDIR</code>	Sets the working folder inside the container; later commands run from here.
<code>COPY</code>	Copies files from your machine into the image. Copying <code>requirements.txt</code> first lets Docker cache the install step.
<code>RUN</code>	Runs a command at build time , such as installing packages.
<code>EXPOSE</code>	Documents the port the app listens on. It doesn't publish it; <code>ports:</code> or <code>-p</code> does.
<code>CMD</code>	The command that runs when the container starts .

Make sure your app listens on `0.0.0.0`, not `127.0.0.1`, or it won't be reachable from outside the container.

Step 2: Write `.dockerignore`

Keeps junk and secrets out of the image and speeds up builds:

```
.git
.gitignore
venv/
.venv/
__pycache__/
*.pyc
.env
.vscode/
.idea/
.DS_Store
```

Step 3: Write `docker-compose.yml`

Defines the web service, the database service, ports, and environment variables:

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - APP_ENV=development
      - DATABASE_URL=postgresql://postgres:${POSTGRES_PASSWORD}@db:5432/mydb
    depends_on:
      - db
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=mydb
    volumes:
      - db_data:/var/lib/postgresql/data
```

```
restart: unless-stopped
```

```
volumes:
  db_data:
```

Secrets live in a separate `.env` file (never committed):

```
POSTGRES_PASSWORD=changeme
```

Key	Meaning
<code>web / db</code>	Service names. They also act as hostnames on Compose's private network, so the app connects to the database at <code>db</code> , not <code>localhost</code> .
<code>build: .</code>	Build the image from the local Dockerfile.
<code>image:</code>	Use a ready-made image from Docker Hub.
<code>ports</code>	"HOST:CONTAINER" port mapping.
<code>environment</code>	Environment variables passed to the container. <code>\${VAR}</code> is read from <code>.env</code> .
<code>depends_on</code>	Start <code>db</code> before <code>web</code> .
<code>volumes</code>	Keeps database data after containers are removed.
<code>restart</code>	<code>unless-stopped</code> restarts the service after crashes or reboots.

Step 4: Launch

```
docker compose up -d
```

Check that everything is healthy:

```
docker compose ps
```

Then open `http://localhost:8000` in your browser.



4. Workflow 2: Cloning & Running an Existing Docker Project From GitHub

Step 1: Install Docker

Install Docker Desktop (Windows/macOS) or Docker Engine with the Compose plugin (Linux), and make sure it's running. Verify:

```
docker compose version
```

Step 2: Clone the repository

```
git clone https://github.com/<username>/<repo>.git
```

Step 3: Enter the project folder

```
cd <repo>
```

Step 4: Create your environment file

Secrets are never on GitHub. If the repo has a `.env.example`, copy it and fill in real values:

```
cp .env.example .env
```

Step 5: Build and start everything

```
docker compose up -d
```

If the project has never been built on your machine, Compose builds it automatically. If you pull changes later, use `docker compose up -d --build`.

Step 6: Verify and use it

```
docker compose ps
```

```
docker compose logs
```

Open the port listed in `docker-compose.yml` (for example `http://localhost:8000`).

Step 7: Shut down when finished

```
docker compose down
```

Why you don't need local Python or database installs

- The **Dockerfile** defines the exact Python version and libraries, installed inside the image.
 - The **docker-compose.yml** pulls and runs the database (PostgreSQL, MySQL, Redis, etc.) as its own container.
 - Your machine only needs **Docker**. No Python, no pip, no database server, and no version conflicts with other projects.
 - Everyone on the team runs the identical environment, which ends "works on my machine" problems.
 - Cleanup is simple: `docker compose down` and your system stays untouched.
-

5. GitHub vs Docker File Rules (`.gitignore` vs `.dockerignore`)

	<code>.gitignore</code>	<code>.dockerignore</code>
Controls	What Git tracks and uploads to GitHub	What Docker copies into the image during build
Goal	Clean repo, no secrets	Small, fast, safe images
Lives in	Project root	Project root (next to the Dockerfile)

What stays on GitHub (commit it)

- Source code
- `Dockerfile`
- `docker-compose.yml`
- `.dockerignore` and `.gitignore`
- `requirements.txt` (or `package.json`)
- `.env.example` with placeholder values
- `README.md`

What gets ignored

Item	.gitignore	.dockerignore	Reason
<code>.env</code> (real secrets)	✓	✓	Never publish or bake in passwords and API keys
<code>venv/</code> , <code>.venv/</code>	✓	✓	Local virtual environment; the container has its own packages
<code>__pycache__/</code> , <code>*.pyc</code>	✓	✓	Generated files
<code>.git/</code>	n/a	✓	Git history is useless and heavy inside an image
<code>.vscode/</code> , <code>.idea/</code> , <code>.DS_Store</code>	✓	✓	Personal editor and OS files
<code>Dockerfile</code> , <code>docker-compose.yml</code>	✗ (commit them)	✓ (optional)	Needed on GitHub, but not needed inside the image
Local DB files, logs	✓	✓	Runtime data, not source code

Golden rules:

1. Never commit `.env` . Commit `.env.example` instead.
2. Always commit the `Dockerfile` and `docker-compose.yml` so anyone can run the project.
3. Keep `.dockerignore` tight so builds are fast and images stay small.